```
        .attr("class", "data")
        .html(function (d) {return d})
        .style("left", function(d,i) {return (i * 100) + "px";});

      d3.select("div.table")
      .selectAll("div.datarow")
      .data(incData, function(d) {return d.content}).enter()
      .append("div")
      .attr("class", "datarow row")
      .style("top", function(d,i) {return (40 + (i * 40)) + "px";});

      d3.selectAll("div.datarow")
      .selectAll("div.data")
      .data(function(d) {return d3.entries(d);})
      .enter()
      .append("div")
      .attr("class", "data")
      .html(function (d) {return d.value})
      .style("left", function(d,i,j) {return (i * 100) + "px";});

    };
```

Notice that `createSpreadsheet` uses a variable `targetDiv` that's passed to each select function. This is in contrast to earlier implementations where the CSS selector was explicitly declared in the code. This way the functions could be reused for different pages.

9.1.3 Bar chart

The second chart we'll add to our dashboard is a bar chart, which requires that we nest the data so that we can count tweets. Because we'll redraw the graphical elements in a separate function, you'll see in the following listing that the code doesn't declare any graphical aspects of the rectangles that make up our bar chart.

Listing 9.6 Bar chart code

```
function createBar(incData,targetSVG) {

   d3.select(targetSVG).selectAll("rect").data(incData)
   .enter()
   .append("rect")
   .attr("class", "bar");
};
```

When we use the redraw function later, we'll use D3 functions that you haven't seen before to draw this chart more effectively and dynamically.

9.1.4 Circle pack

You learned how to make a circle pack chart in chapter 5. Listing 9.7 shows how we can deploy similar code to lay the groundwork for adding our circle pack chart to our dashboard. As with the bar chart, we don't declare any of the graphical channels that we expect to change if the page is resized, which means we only declare fill and stroke.